
Applied Reinforcement Learning

Agent Architectures Exercises

Riccardo Berta

2026.09.17

0.1 Exercises on Agent Architectures

These activities reinforce the core ideas presented in the Agent Architectures chapter: the PEAS framework, the distinction between the agent function and the agent program, why table-driven agents are impractical, the progression from reflex to model-based, goal-based, and utility-based agents, rationality as expected utility maximization, and the boid model as a concrete example of a state-update architecture.

0.1.1 Exercise 1.1 — PEAS and environment classification

Consider an autonomous delivery robot that drives around a warehouse, picks up parcels from shelves, and drops them at packing stations, sharing the floor with human workers and other robots.

1. Give a PEAS description (Performance measure, Environment, Actuators, Sensors)
2. Classify the task environment along the six axes: observable, single/multi-agent, deterministic/stochastic, static/dynamic, discrete/continuous, known/unknown, justifying each choice in one line.

Step 1 — PEAS description

Component	Specification
Performance	parcels delivered correctly per hour; time/energy per delivery; number of collisions or near-misses (penalised)
Environment	warehouse floor, shelves, parcels, packing stations, human workers, other robots
Actuators	drive motors (wheels), steering, gripper/lift arm, signalling lights/speaker
Sensors	cameras, LIDAR/range finders, wheel odometry, gripper force/contact sensors, battery gauge

Step 2 — Environment classification

Axis	Answer	Justification
Observable	Partially	on-board sensors see only the local surroundings, not the whole warehouse
Agents	Multi-agent	humans and other robots act and their behaviour affects the robot
Deterministic	Stochastic	wheel slip, sensor noise, and unpredictable humans make outcomes uncertain
Static / Dynamic	Dynamic	the world changes while the robot deliberates (people move)
Discrete / Continuous	Continuous	positions, velocities and time vary smoothly
Known / Unknown	Partially known	physics of driving is known; the live positions of people/parcels must be learned online

Key concept

PEAS forces us to make the performance measure explicit and measurable before designing the agent, and the six axes tell us how hard the environment is, here the hardest combination (partially observable, multi-agent, stochastic, dynamic, continuous), exactly like autonomous driving.

0.1.2 Exercise 1.2 — Size of the lookup table

A table-driven agent stores one action for every possible percept sequence. Let P be the number of distinct percepts and let the agent live for T time steps (so percept sequences have length 1 to T).

1. Derive a closed-form expression for the number of table entries E
2. Evaluate it for the vacuum world, where a percept is a (location, status), so $P=4$, and $T=3$

3. Comment on why this makes table-driven design impossible in general.

Step 1 — Count the sequences

There are P^t distinct percept sequences of length exactly t (each of the t positions can be any of P percepts). Summing over all lengths $t = 1, \dots, T$:

$$E = \sum_{t=1}^T P^t$$

This is a finite geometric series with ratio P . Using the formula for the sum of a geometric series:

$$\sum_{t=1}^T x^t = \frac{x(x^T - 1)}{x - 1}$$

we get the closed form:

$$E = \frac{P(P^T - 1)}{P - 1}$$

Step 2 — Evaluate

With $P=4$ and $T=3$ and using the sum formula:

$$E = 4 + 4^2 + 4^3 = 4 + 16 + 64 = 84$$

or via the closed form:

$$E = \frac{4(4^3 - 1)}{4 - 1} = \frac{4 \cdot 63}{3} = 84$$

Step 3 — Interpret

E grows exponentially in the horizon T . Even this toy vacuum world already needs 84 entries for only three steps, the driving example (a single camera, one hour) yields on the order of several million percepts per second, so a 10-minute episode is clearly impossible to store, and even if it were, the agent would have no way to learn the correct action for each entry.

Key concept

The agent function (percept-sequence $\rightarrow$ action) is a well-defined mathematical object, but representing it explicitly as a table is impossible. This is exactly why we need a compact agent program (e.g. a reflex rule) that computes the same mapping without storing it.

0.1.3 Exercise 1.3 — Why a reflex agent may need randomisation?

A vacuum agent can perceive only whether its current square is clean or dirty, it cannot determine whether it is currently in square A or square B. Assume that both squares are clean. A deterministic reflex agent must therefore map the percept clean to the same action every time.

1. Show that, in this partially observable environment, every deterministic reflex program can fail, causing the agent to loop indefinitely without reaching the other square.
2. Now consider a randomised reflex agent that, whenever it perceives clean, chooses Left or Right independently, each with probability $1/2$. Model the number of actions required for the agent to reach the other square as a random variable, determine its probability distribution, and compute its expected value.

Step 1 — Deterministic failure

With only the percept "clean", a deterministic rule fixes one action, say always "right". If the agent starts in the rightmost square, "right" bumps against the wall and it stays put; it perceives "clean" again and repeats forever: an infinite loop. Symmetrically for "left". No deterministic reflex rule works from both squares because the agent cannot distinguish them.

Step 2 — Randomised agent

Now suppose that whenever the agent perceives clean, it chooses randomly between Left and Right, with probability $1/2$ for each action. Consider the agent while it is in one of the two squares. Exactly one of the two actions moves it to the other square, the other action points toward the wall, so the agent bumps into it and remains where it is. Therefore, at every step,

$$p = \Pr(\text{reach the other square}) = \frac{1}{2}$$

while

$$1 - p = \Pr(\text{bump into the wall and remain here}) = \frac{1}{2}$$

The important observation is that after a failed attempt, nothing has changed: the agent is still in the same square, still perceives clean, and again chooses Left or Right with equal probability. Thus, every step is another independent attempt with the same probability $p=1/2$ of success.

Let N be the number of steps needed to reach the other square. The agent succeeds on each step with probability $p=0.5$. Therefore:

$$P(N = 1) = p$$

$$P(N = 2) = (1 - p)p$$

$$P(N = 3) = (1 - p)^2 p$$

$$P(N = 4) = (1 - p)^3 p$$

and so on

With $p=0.5$:

$$P(N = 1) = 1/2$$

$$P(N = 2) = 1/4$$

$$P(N = 3) = 1/8$$

$$P(N = 4) = 1/16$$

...

Thus, there is a 50% chance of reaching the other square immediately, a 25% chance of requiring exactly two actions, a 12.5% chance of requiring exactly three, etc. In general:

$$P(N = k) = (1 - p)^{k-1}p$$

The expected number of steps can be derived directly from the structure of the problem. Let $E[N]$ denote the expected number of steps needed to reach the other square. The agent always takes one step. With probability p , that step succeeds and the process ends. With probability $1-p$, the agent hits the wall and remains in the same square. In that case, after spending one step, it is back in exactly the same situation as before and still needs, on average, another $E[N]$ steps. Therefore,

$$E[N] = p \cdot 1 + (1 - p)(1 + E[N])$$

$$\text{Since } p + (1 - p) = 1$$

$$E[N] = 1 + (1 - p)E[N]$$

and hence

$$pE[N] = 1$$

so

$$E[N] = \frac{1}{p}$$

With $p=0.5$,

$$E[N] = \frac{1}{0.5} = 2$$

So the randomised agent reaches the other square in two actions on average. Notice that this does not mean it always takes two actions: sometimes it succeeds immediately, sometimes it takes two, three, ten, or more. In fact, there is no finite upper bound on N , but the probability of very long runs decreases exponentially.

Key concept

Randomisation resolves the perceptual aliasing created by partial observability. Because A and B produce the same percept, a deterministic reflex policy cannot choose different actions in the two states. Randomisation does not give the agent any additional information, but it prevents it from being permanently trapped by always making the same choice. This is a useful early example of how stochastic action selection can compensate, to some extent, for limited information.

0.1.4 Exercise 1.4 — Rational action = maximising expected utility

A utility-based agent must choose between two actions in an uncertain environment. The designer's utility function U assigns the following utilities to the possible outcomes:

Action A1 (risky): with probability 0.8, the outcome has utility $U=+10$, with probability 0.2, it has utility $U=-5$.

Action A2 (safe): the outcome is certain and has utility $U=+6$.

1. Compute the expected utility of each action. According to the principle of rationality, which action should the agent choose?
2. Now suppose an omniscient agent knows the actual outcome in advance and knows that, on this particular occasion, choosing A1 would produce the outcome with utility -5. Which action would it choose?
3. Explain why the omniscient agent's choice does not imply that choosing A1 was irrational for the original agent. What does this tell us about the difference between rationality and omniscience?

Step 1 — Compute the expected utility

Because the environment is uncertain, the agent cannot know which outcome will actually occur when it chooses an action. Instead, it evaluates each action by considering all possible outcomes, weighting each utility by the probability that the corresponding outcome will occur:

$$E[U|a] = \sum_{\text{outcomes}} \Pr(\text{outcome} | a) U(\text{outcome})$$

For the risky action A1, there are two possible outcomes:

$$E[U|A1] = 0.8(10) + 0.2(-5) = 8 - 1 = 7$$

Intuitively, if the agent could face the same decision many times under the same conditions, choosing A1 would yield an average utility of 7 per decision in the long run. This does not mean that any individual choice of A1 produces utility 7: each time, the actual utility will be either +10 or -5.

For the safe action A2, there is no uncertainty:

$$E[U|A2] = 1.0(6) = 6$$

Since $E[U|A1] > E[U|A2]$, the rational choice is A1. Notice that the agent is not choosing A1 because it is certain to produce the better outcome (it is not). It chooses A1 because, given the information currently available, it offers the highest expected utility.

Step 2 — Rationality is not omniscience

Now suppose that an omniscient agent somehow knows in advance that, on this particular occasion, A1 will produce the bad outcome with utility -5. Its decision is then straightforward, since $U(A1)=-5$ and $U(A2)=+6$, it would therefore choose A2.

This does not mean that choosing A1 was irrational for the original agent. The two agents are making their decisions with different information. The original agent knows only the probabilities 0.8 and 0.2, whereas the omniscient agent knows which outcome will actually occur.

Rationality must therefore be evaluated using the information available at the time the decision is made, not by looking retrospectively at what happened afterward.

For example, suppose the original agent rationally chooses A1 and happens to receive -5. We should not conclude: "The outcome was bad, therefore the decision was irrational". The decision was still rational because A1 had the highest expected utility when the decision was made. A rational decision can produce a bad outcome when the environment is uncertain. Conversely, an irrational decision can sometimes produce a good outcome simply through luck. The quality of a decision and the quality of its eventual outcome are therefore not the same thing.

Key concept

A rational utility-based agent chooses the action that maximises expected utility given the information available at decision time. It does not necessarily choose the action that will ultimately produce the highest actual utility, because knowing that would require omniscience.

0.1.5 Exercise 1.5 — One step of a boid (Euler integration)

A boid (model-based agent) has mass $m=1$ and, at time t , position $p(t)=(0,0)$ and velocity $v(t)=(1,0)$. Its three steering rules produce the (unit) force vectors:

$$F_{\text{sep}} = (1, 0)$$

$$F_{\text{coh}} = (0, -1)$$

$$F_{\text{align}} = (1, 1)$$

combined with weights:

$$w_{\text{sep}} = 1.2$$

$$w_{\text{coh}} = 0.5$$

$$w_{\text{align}} = 1.0$$

Using the explicit Euler rule with $\Delta t = 1$:

$$v(t + \Delta t) = v(t) + a(t) \Delta t$$

$$p(t + \Delta t) = p(t) + v(t) \Delta t$$

$$a = \frac{F}{m}$$

compute the boid's state after one update (and, as a check, after a second update).

Step 1 — Combine the steering forces

The three steering rules do not act independently. Their contributions are combined into a single net force, with the weights determining how strongly each rule influences the boid:

$$\mathbf{F} = w_{\text{sep}}\mathbf{F}_{\text{sep}} + w_{\text{coh}}\mathbf{F}_{\text{coh}} + w_{\text{align}}\mathbf{F}_{\text{align}}$$

Substituting the given values,

$$\mathbf{F} = 1.2(1, 0) + 0.5(0, -1) + 1.0(1, 1) = (1.2 + 1.0, -0.5 + 1.0) = (2.2, 0.5)$$

Computing the two components separately,

$$F_x = 1.2 + 0 + 1.0 = 2.2$$

$$F_y = 0 - 0.5 + 1.0 = 0.5$$

The result also has a simple geometric interpretation: separation and alignment both push the boid to the right, while cohesion pulls downward and alignment pushes upward. The latter two effects partially cancel, leaving a smaller upward component.

Step 2 — Convert force into acceleration

From Newton's second law,

$$\mathbf{a} = \frac{\mathbf{F}}{m}$$

Since $m=1$,

$$\mathbf{a} = (2.2, 0.5)$$

Thus, during this time step, the steering rules cause the boid to accelerate strongly to the right and slightly upward.

Step 3 — Apply the first Euler update

At time t , the state is

$$\mathbf{p}(t) = (0, 0), \quad \mathbf{v}(t) = (1, 0)$$

and $\Delta t = 1$.

First, update the velocity using the current acceleration:

$$\mathbf{v}(t+1) = \mathbf{v}(t) + \mathbf{a}(t)\Delta t = (1, 0) + (2.2, 0.5)(1) = (3.2, 0.5)$$

Next, update the position using the current velocity:

$$\mathbf{p}(t+1) = \mathbf{p}(t) + \mathbf{v}(t)\Delta t = (0, 0) + (1, 0)(1) = (1, 0)$$

Hence, after one update, the boid's new state is

$$\mathbf{p}(t+1) = (1, 0), \quad \mathbf{v}(t+1) = (3.2, 0.5)$$

The important detail is that explicit Euler uses the old velocity to update the position. Although the acceleration has already changed the velocity to $(3.2, 0.5)$, that new velocity affects the position only during the next time step.

Step 4 — Check with a second update

As a numerical check, suppose that the steering forces remain unchanged during the second step, so that

$$\mathbf{a}(t+1) = (2.2, 0.5)$$

Starting from

$$\mathbf{p}(t+1) = (1, 0), \quad \mathbf{v}(t+1) = (3.2, 0.5)$$

the velocity becomes

$$\mathbf{v}(t+2) = (3.2, 0.5) + (2.2, 0.5) = (5.4, 1.0)$$

while the position becomes

$$\mathbf{p}(t+2) = (1, 0) + (3.2, 0.5) = (4.2, 0.5)$$

Therefore, after the second update,

$$\mathbf{p}(t+2) = (4.2, 0.5), \quad \mathbf{v}(t+2) = (5.4, 1.0)$$

In an actual boid simulation, however, the acceleration would normally not remain constant. After the first movement, the boid has a new position and velocity relative to its neighbours, so separation, cohesion, and alignment must be computed again. Keeping the same acceleration here is only a convenient check of the Euler calculations.

Key concept

A boid repeatedly applies a simple local perception–action–update cycle. Each boid responds only to local information; there is no global controller specifying the trajectory of the flock. Nevertheless, repeatedly applying these simple local rules across many interacting boids can produce coordinated, flock-like behavior.